컬렉션 자료형

학습목표

- 파이썬에서 컬렉션 자료형을 설명할 수 있다.
- 리스트와 튜플을 활용하여 문제를 해결할 수 있다.
- 딕셔너리를 활용하여 문제를 해결할 수 있다.
- 세트를 활용하여 문제를 해결할 수 있다.
- 컬렉션 자료형의 특징과 차이점을 이해하여 효율적인 자료형을 선택하여 문제를 해결할 수 있다.

목차

5.1 컬렉션 자료형 개념과 필요성
5.2 리스트 자료형
5.3 튜플 자료형
5.4 딕셔너리 자료형
5.5 세트 자료형
5.6 컬렉션 자료형 실습
5.7 컬렉션 자료형 도전문제

5.1 컬렉션 자료형 개념과 필요성

- 자료형이란 프로그래밍을 할 때 쓰이는 숫자, 문자열 등 자료 형태로 사용하는 모든 것을 뜻한다.
- "컬렉션"이란? 여러 개의 값을 하나의 변수에 담을 수 있으며 변수 안에 공간을 여러 개 가지며, 변수 안에 서로 다른 공간을 찾는 방법이 있다.
- 파이썬에서 제공하는 컬렉션 자료형의 종류는 다음과 같다.

표 5.1

컬렉션 자료형 종류	생성 방법
리스트(list)	[값1, 값2, ...]
튜플(tuple)	(값1, 값2, ...)
딕셔너리(dictionary)	{키:값, ...}
세트(set)	{값1, 값2, ...}

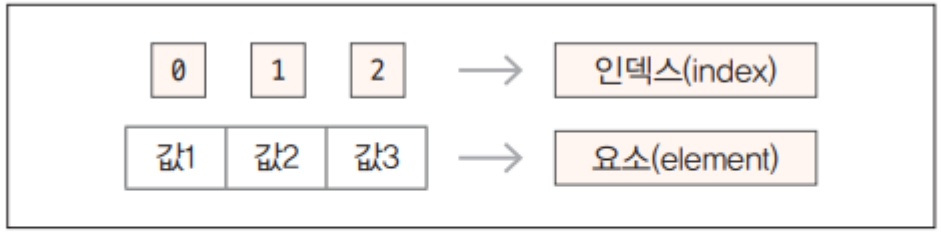
5.2 리스트 자료형

- 파이썬 프로그래밍 언어 내에서 가장 많이 쓰이는 구조인 리스트를 이용하면 많은 양의 데이터들을 한 번에 모아 효율적으로 처리하고 저장할 수 있다.
- 기존의 변수는 오직 하나의 데이터를 저장하기 때문에 1000명의 점수 데이터를 저장하기 위해서는 1000개 의 변수가 필요하다. 반면에 리스트는 오직 하나의 리스트 변수를 이용하여 1000명의 점수 데이터를 효율적으로 저장하고 처리할 수 있다.

5.2 리스트 자료형

(1) 리스트 문법

- 대괄호[] 안에 서로 다른 자료형의 값을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형



리스트 문법

```
리스트명 = [값1, 값2, 값3, .....]
```

5.2 리스트 자료형

- 빈 리스트 생성과 리스트 초기화

빈 리스트 생성

```
>>> a = []
>>> a
[]
>>> b = list()
>>> b
[]
```

리스트 초기화

```
>>> a
[1, 2, 3, 4, 5]
>>> b=['Hello','Python']
>>> b
['Hello', 'Python']
>>> c=[1,2,'Hello',[10,20]]
>>> c
[1, 2, 'Hello', [10, 20]]
```

5.2 리스트 자료형

(2) 리스트 인덱싱(Indexing) 및 슬라이싱(Slicing)

- 리스트도 문자열처럼 인덱싱과 슬라이싱이 가능하다.
- 인덱싱(Indexing)은 무엇인가 '가르킨다'라는 의미이고 슬라이싱(Slicing)은 무엇인가 '잘라낸다'라는 의미이다.

리스트 인덱스(index) 구조

```
>>> a=[10, 20, 30, 40]
```

인덱스(index) →	0	1	2	3
	10	20	30	40
인덱스(index) →	-4	-3	-2	-1

5.2 리스트 자료형

- 리스트 인덱싱(Indexing)

리스트 인덱싱(Indexing) 사용하기

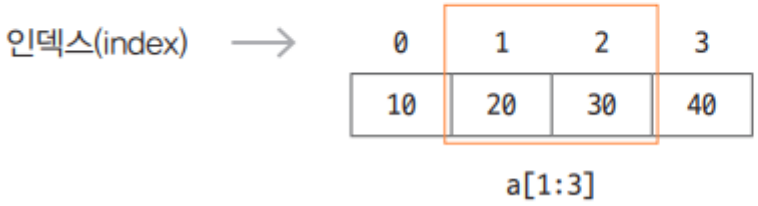
```
>>> a=[10,20,30,40]
>>> a
[10, 20, 30, 40]
>>> a[0] # 리스트 첫 번째 값을 가져온다.
10
>>> a[0]+a[1]
30
>>> a[-1] # 리스트 마지막 값을 가져온다.
40
>>> a[4]
Traceback (most recent call last):
  File "<pyshell#21>", line 1, in <module>
    a[4]
IndexError: list index out of range
```

5.2 리스트 자료형

- 리스트 슬라이싱(Slicing)

리스트 슬라이싱

```
>>> a=[10,20,30,40]
>>> a[1:3] # 슬라이싱은 새로운 리스트를 반환한다.
[20,30]
```



슬라이싱은 리스트 안에서 범위를 지정하여 원하는 요소들을 선택하는 연산으로 리스트명[시작:끝]와 같은 형식을 사용하고 시작요소부터 (끝-1) 인덱스에 있는 요소까지 선택한다.

5.2 리스트 자료형

- 리스트 슬라이싱(Slicing)

리스트 슬라이싱(Slicing) 사용하기

```
>>> a=[10,20,30,[40,50]]
>>> a[0:2]
[10, 20]
>>> a[1:]
[20, 30, [40, 50]]
>>> a[:2]
[10, 20]
>>> a[-1]
[40, 50]
>>> a[3]
[40, 50]
>>> a[3][0] # 이중 리스트 슬라이싱 하기
40
>>> a[3][1] # 이중 리스트 슬라이싱 하기
50
```

5.2 리스트 자료형

(3) 2차원 리스트의 기초

- 파이썬에서 2차원 리스트는 리스트의 리스트로 표현됩니다. 이를 통해 행과 열을 가지는 데이터를 구조 화할 수 있습니다. 2차원 리스트는 행(row)과 열(column)로 이루어진 데이터 테이블처럼 사용할 수 있습니다

2차원 리스트 생성 및 접근

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
# 첫 번째 행, 두 번째 열의 요소에 접근
element = matrix[0][1]  # 2
print(element)
```

2차원 리스트의 요소 수정

```
matrix[1][2] = 10
# 결과: [[1, 2, 3], [4, 5, 10], [7, 8, 9]]
```

5.2 리스트 자료형

(4) 리스트 값 변경하기

- 리스트의 값을 변경하기 위해서는 첨자(인덱스)를 사용해서 그 위치의 값을 변경하면 된다.

리스트 값 변경하기

```
>>> a=[10,20,30]
>>> a
[10, 20, 30]
>>> a[0]=100
>>> a
[100, 20, 30]
>>> a[1:3]=[90,100] # 연속된 범위의 값도 변경 가능하다.
>>> a
[100, 90, 100]
```

5.2 리스트 자료형

(4) 리스트 조작함수

- 문자열과 마찬가지로 리스트도 리스트변수 이름 뒤에 마침표(.)를 붙인 다음 함수 이름을 사용하면 된다.

리스트에 값 추가(append)

```
>>> a=[10,20]
>>> a.append(30)
>>> a
[10, 20, 30]
>>> a.append([40,50]) # 리스트에 리스트도 추가 가능
>>> a
[10, 20, 30, [40, 50]]
>>> a.insert(1,15)
>>> a
[10, 15, 20, 30, [40, 50]]
```

- append(x) : 리스트 맨 마지막에 x의 값을 추가한다.
- insert(index, x) : 리스트의 원하는 위치에 값을 추가(삽입)할 수 있다. 리스트 위치 (index)에 x의 값을 넣는다.

5.2 리스트 자료형

리스트 정렬(sort)

```
>>> a=[8,6,5,2,4]
>>> a.sort() # 오름차순 정렬
>>> a
[2, 4, 5, 6, 8]
>>> a.sort(reverse=True) # 내림차순 정렬
>>> a
[8, 6, 5, 4, 2]
>>> a=['haha', 'aa', 'pupu']
>>> a.sort() # 알파벳 정렬
>>> a
['aa', 'haha', 'pupu']
>>> a=['리스트', '튜플', '딕셔너리', '세트']
>>> a.sort() # 한글 정렬
>>> a
['딕셔너리', '리스트', '세트', '튜플']
>>> a.sort(reverse=True)
>>> a
['튜플', '세트', '리스트', '딕셔너리']
```

5.2 리스트 자료형

리스트에 값 삭제(pop과 remove)

```
>>> a=[10,20,30,40]
>>> a.pop()
40
>>> a
[10, 20, 30]
>>> a.pop(1)
20
>>> a
[10, 30]
>>> a.remove(30)
>>> a
[10]
>>> a.remove(0)
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    a.remove(0)
ValueError: list.remove(x): x not in list
```

- pop() : 리스트 맨 뒤의 값을 빼내고, 빼낸 항목은 삭제한다.
- pop(index) : 리스트에서 index가 가리키는 값을 빼내고, 빼낸 항목은 삭제한다.
- remove(x) : 리스트에서 x를 삭제하는 함수이다.(단, x값이 중복되면 첫 번째만 삭 제한다.)

5.2 리스트 자료형

리스트에서 찾을 값의 개수 세기(count)

```
>>> a=[10,20,30,20]
>>> a.count(20)
2
>>> a=['박수지','배수지','배수지']
>>> a.count('배수지')
2
```

리스트에서 값의 위치 반환(index)

```
>>> a=[10,20,30]
>>> a.index(20)
1
>>> a.index(10)
0
>>> a.index(40)
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    a.index(40)
ValueError: 40 is not in list
```

5.2 리스트 자료형

표 5.3 리스트 주요 조작 함수

함수	설명	사용법
append()	리스트에 요소를 마지막 위치에 새로 추가	리스트.append(값)
insert()	리스트의 해당 위치에 요소를 새로 삽입	리스트.insert(위치, 값)
sort()	- 오름차순 정렬 - 내림차순 정렬	리스트.sort() 리스트.sort(reverse=True)
reverse()	현재의 리스트를 그대로 거꾸로 뒤집는다.	리스트.reverse()
pop()	- 리스트 제일 뒤의 항목을 빼내고 빼낸 항목은 삭제 - 제거할 위치에 있는 요소를 제거	리스트.pop() 리스트.pop(위치)
remove()	해당 요소를 찾아 삭제	리스트.remove(삭제할 값)
count()	해당 요소의 개수를 반환	리스트.count(찾을 값)
index()	리스트에 위치값이 있으면 위치값을 반환	리스트.index(값)
len()	리스트 총 요소 개수를 반환	len(리스트)

5.3 튜플 자료형

- 리스트는 프로그램 실행 중에 바뀔 수 있는 요소를 저장하는 데 알맞은 자료형이다.
- 튜플은 한번 저장된 값은 수정할 수 없는 자료형으로 읽기 전용의 데이터를 저장할 때 유용하게 사용된다.
- 프로그램이 실행되는 동안 데이터가 항상 변하지 않아야 한다면 튜플을 사용하고 이와는 반대로 수시로 데이터를 변화시켜야 할 경우라면 리스트를 사용해야 한다.

5.3 튜플 자료형

(1) 튜플의 문법

- () 안에 서로 다른 자료형의 값을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형이다.
- 0부터 시작하는 인덱스를 이용해 접근할 수 있고 한 번 저장된 요소는 변경 할 수 없다.

튜플 문법

튜플명 = (값1, 값2, 값3, ...)

5.3 튜플 자료형

(2) 튜플의 인덱싱(Indexing) 및 슬라이싱(Slicing)

- 튜플은 저장된 데이터를 변경시킬 수 없다는 점만 제외하면 리스트와 완전히 동일하다.

튜플 인덱싱(Indexing)과 슬라이싱(Slicing)

```
>>> a=(10,20,30,40)
>>> a[0]
10
>>> a[-1]
40
>>> a[1:]
(20, 30, 40)
>>> a[:2]
(10, 20)
```

5.4 딕셔너리 자료형

- 딕셔너리(dictionary)는 한글로 표현하면 '사전'이다.
- 한글, 영어 사전들은 가나다순 알파벳순으로 정렬 하게 된다.
- 딕셔너리는 순서가 없는 컬렉션 자료형으로 각각의 요소는 key:value 형태로 저장된다.
- 딕셔너리는 리스트나 튜플처럼 index에 의해 해당 요소를 찾지 않고 key를 통해 value를 얻는다. 이것이 바로 딕셔너리의 가장 큰 특징이다.

5.4 딕셔너리 자료형

(1) 딕셔너리 문법

- {}안에 키:값 형식의 항목을 콤마(,)로 구분해 하나 이상 저장할 수 있는 컬렉션 자료형이다.
- 딕셔너리는 키를 먼저 지정하고 :(콜론)을 붙여서 값을 표현한다. 키와 값은 1:1 대응관계이다.

딕셔너리 구조

딕셔너리 문법

딕셔너리 명 = {key1:value1, key2:value2, key3:value3,...}

5.4 딕셔너리 자료형

빈 딕셔너리 생성

```
>>> a={}
>>> a
{}
>>> a=dict()
>>> a
{}
```

딕셔너리 초기화

```
>>> fruit={1:'사과',2:'딸기'}
>>> fruit
{1: '사과', 2: '딸기'}
>>> menu={'김밥':2000,'라면':3000}
>>> menu
{'김밥': 2000, '라면': 3000}
```

5.4 딕셔너리 자료형

(2) 딕셔너리 요소에 접근하기

- 딕셔너리를 생성할 때는 중괄호를 사용하지만 딕셔너리 요소에 접근할 때는 리스트처럼 딕셔너리 뒤에 대괄호[]를 입력하고 내부에 인덱스 대신 키를 입력한다.

딕셔너리 요소 접근

```
>>> menu={'김밥':2000,'라면':3000}
>>> menu['김밥']
2000
>>> menu['라면']
3000
>>> menu['라면']=3500
>>> menu['라면']
3500

>>> menu['어묵']=1000
>>> menu
{'김밥': 2000, '라면': 3500, '어묵': 1000}
>>> menu['떡볶이']
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    menu['떡볶이']
KeyError: '떡볶이'
```

5.4 딕셔너리 자료형

(3) 딕셔너리 조작함수

모든 key, value, item 반환

```
>>> menu={'김밥': 2000, '라면': 3500, '어묵': 1000}
>>> menu.keys()
dict_keys(['김밥', '라면', '어묵'])
>>> menu.values()
dict_values([2000, 3500, 1000])
>>> menu.items()
dict_items([('김밥', 2000), ('라면', 3500), ('어묵', 1000)])
```

- keys() 함수 : 딕셔너리의 키들만 모아서 반환한다.
- values() 함수 : 딕셔너리의 값들만 모아서 반환한다.
- items() 함수 : Key와 Value의 쌍을 튜플로 묶은 값을 반환한다.

5.4 딕셔너리 자료형

(3) 딕셔너리 조작함수

key로 value 얻기: get() 함수

```
>>> name={100:'황복동',200:'황채연',300:'황나연'}
>>> name[100]
'황복동'
>>> name[400]
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    name[400]
KeyError: 400
>>> name.get(100)
'황복동'
>>> name.get(400,'not Found')
'not Found'
```

- get() 함수 : 딕셔너리의 키로 값을 추출하는 기능으로 딕셔너리[키]와 같은 기능을 수행하지만, 키가 존재하지 않을 경우에는 KeyError를 발생키지 않는다.

5.4 딕셔너리 자료형

(3) 딕셔너리 조작함수

딕셔너리 요소 삭제하기: del() 함수, pop() 함수

```
>>> name={100:'황복동',200:'황채연',300:'황나연'}
>>> del(name[100])
>>> name
{200: '황채연', 300: '황나연'}
>>> name.pop(200)
'황채연'
>>> name
{300: '황나연'}
```

5.4 딕셔너리 자료형

표 5.4 딕셔너리 조작 함수

함수	설명	사용법
get()	항목 접근하기	딕셔너리.get(key)
pop()	항목 꺼내고 삭제하기	딕셔너리.pop(key)
del()	항목 삭제하기	del(딕셔너리[key])
items()	딕셔너리에 저장된 항목	딕셔너리.items()
keys()	딕셔너리에 저장된 키	딕셔너리.keys()
values()	딕셔너리에 저장된 값	딕셔너리.values()

5.5 세트 자료형

- 세트(set) 자료형은 집합에 관련된 것을 쉽게 처리하기 위해 만든 자료형으로 중복을 허용하지 않는 컬렉션 자료형이다.
- 리스트나 튜플은 순서가 있기 때문에 인덱싱을 통해 자료형의 값을 얻을 수 있지만 세트 자료형은 순서가 없기 때문에 인덱싱으로 값을 얻을 수 없다.
- 세트 자료형의 특징은 다음과 같다.
    - 수학의 집합과 같다.
    - 중복되지 않은 요소들로 구성된다.
    - 세트간의 순서가 없다.

5.5 세트 자료형

(1) 세트의 문법

세트 문법

```
세트명 = set() # 비어 있는 세트 생성
```

세트 초기화

```
>>> a={10,20,30}
>>> a
{10, 20, 30}
>>> a={10,20,30,20}
>>> a
{10, 20, 30}
>>> b=set([10,20,30]) # 리스트-> 세트 자료형 변환
>>> b
{10, 20, 30}
```

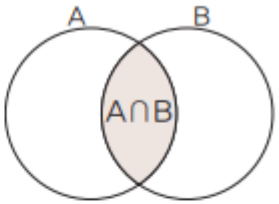
중복을 허용하지 않음!

5.5 세트 자료형

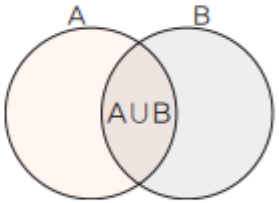
(2) 세트 연산

세트 합집합, 교집합, 차집합

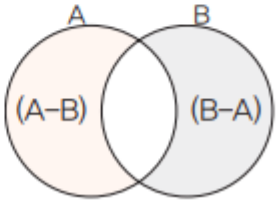
```
>>> A={10,20,30}
>>> B={20,40}
>>> A & B          # 교집합
{20}
>>> A.intersection(B) # 교집합
{20}
>>> A | B          # 합집합  or에 쓰는 |
{20, 40, 10, 30}   중복제거해서 전부 출력
>>> A.union(B)      # 합집합
{20, 40, 10, 30}
>>> A - B          # 차집합
{10, 30}
>>> A.difference(B)  # 차집합
{10, 30}
```



교집합



합집합



차집합

5.5 세트 자료형

(3) 세트 조작 함수

값 추가와 삭제

```
>>> s1={10,20,30}
>>> s1
{10, 20, 30}
>>> s1.add(40)
>>> s1
{40, 10, 20, 30}
>>> s1.update([50,60])
>>> s1
{40, 10, 50, 20, 60, 30}
>>> s1.remove(40)
>>> s1
{10, 50, 20, 60, 30}
>>> s1.clear()
>>> s1
set()
```

- add() 함수 : 값 1개 추가
- update() 함수 : 값 여러 개 추가
- remove() 함수 : 특정 값 삭제

5.6 컬렉션 자료형 실습

1. 다음 파이썬 코드의 괄호 안 결과를 예측해 보시오.

```
>>> color=['red','yellow']
>>> color.append('green')
>>> color
(                              )
>>> color.pop()
(                              )
>>> color.insert(1,'orange')
>>> color
(                              )
>>> color.index('orange')
(                              )
>>> len(color)
(                              )
```

5.6 컬렉션 자료형 실습

2. 다음 파이썬 코드의 괄호 안 결과를 예측해 보시오.

```
>>> score=list(range(1,6))
>>> score
[1, 2, 3, 4, 5]
>>> score[2]
(                              )
>>> score[:3]
(                              )
>>> score[-1]
(                              )
>>> score[2:4]
(                              )
>>> score[3:]
(                              )
```

5.6 컬렉션 자료형 실습

3. 다음 파이썬 코드의 괄호 안 결과를 예측해 보시오.

```
>>> A={1,2,3}
>>> B={3,4,5,5}
>>> A | B
(                                )
>>> A & B
(                                )
>>> A- B
(                                )
```

5.7 컬렉션 자료형 도전문제

1. 다음 파이썬 코드의 실행 결과가 나오도록 괄호 안을 채우시오.

```
>>> spot=['경리단길','가로수길','홍대','대학로']
>>> select=(        )
```

실행결과

```
>>> select
['홍대', '대학로']
```

2. 파이썬 코드를 실행한 결과가 나오게 하기 위해 밑줄 안에 들어갈 명령문을 작성해 보시오.

```
>>> stu1={'Yang','Jang','Oh','Kim'}
>>> stu2={'Hwang','Jang','Kim','Park'}
>>> stu1 __________ stu2
```

실행결과

```
>>>
{'Kim', 'Jang'}
```

5.7 컬렉션 자료형 도전문제

3. 아래와 같은 실행결과가 나오도록 밑줄에 적절한 함수를 채우시오.

```
>>> people={100:'yang',200:'jang',300:'o'}
>>> list(people._____)
[100, 200, 300]
>>> list(people._____)
['yang', 'jang', 'o']
>>> list(people._____)
[(100, 'yang'), (200, 'jang'), (300, 'o')]
>>> people._____(200)
'jang'
>>>_____(people[100])
>>> people
{200: 'jang', 300: 'o'}
```

5.7 컬렉션 자료형 도전문제

4. 아래 문제를 1번부터 7번까지 차례대로 수행하고 단계별 출력결과를 보고 밑줄 안에 알맞은 답을 예측하시오.

문제

1. 학생이 3명(홍일동, 홍이동, 홍삼동)인 과에 홍사동이 편입을 했다. "홍사동"을 리스트에 추가해 보시오.
2. 위 리스트에 동명이인 홍이동이 새로 편입을 했다고 가정하고 요소 "홍이동" 뒤에 "홍이동"을 추가해 보시오.
3. "홍이동"이 몇 명인지 출력하시오.
4. 현재 출석부를 역순으로 출력하시오.
5. "홍일동"이 다른 과로 전과를 하였습니다. "홍일동"을 찾아 삭제하시오.
6. 현재 출석부를 내림차순으로 정렬하시오.
7. 현재 남아 있는 학생의 인원수를 구하시오.

5.7 컬렉션 자료형 도전문제

```
name=['홍일동','홍이동','홍삼동']

print(name)

#1)
_____________________________
print(1,name)

#2)
_____________________________
print(2,name)

#3)
_____________________________

#4)
_____________________________
print(4,name)

#5)
_____________________________
print(5,name)
```

5.7 컬렉션 자료형 도전문제

```
#6)
_____________________________
print(6,name)

#7)
_____________________________
```

실행결과

```
['홍일동', '홍이동', '홍삼동']
1 ['홍일동', '홍이동', '홍삼동', '홍사동']
2 ['홍일동', '홍이동', '홍이동', '홍삼동', '홍사동']
3 2
4 ['홍사동', '홍삼동', '홍이동', '홍이동', '홍일동']
5 ['홍사동', '홍삼동', '홍이동', '홍이동']
6 ['홍이동', '홍이동', '홍삼동', '홍사동']
7 4
```

5.7 컬렉션 자료형 도전문제

5. 채팅 어플에서 친구를 등록하고 친구를 이름으로 검색한 후 연락처를 출력하는 프로그램을 작성하시오. 실행결과를 참고하여 문제를 해결하시오.

조건
1. 딕셔너리 자료구조를 사용하여 친구의 이름과 연락처를 저장합니다(임의대로).
2. 딕셔너리에서 전체 친구의 이름만 검색하여 리스트로 변환한 후 출력합니다.
3. 찾는 친구가 있으면 연락처를 출력하고 없으면 에러 메시지를 출력합니다.
4. 찾는 친구의 이름은 사용자로부터 입력 받아 처리합니다(변수명은 적절하게 사용하시오).

5.7 컬렉션 자료형 도전문제

```
addr={}

addr['최재원']='010-1111-1234'
addr['최지윤']='010-2222-1234'
addr['김연수']='010-3333-1234'
addr['김연우']='010-4444-1234'
addr['김가현']='010-5555-1234'
addr['김혜현']='010-6666-1234'


print(addr)
print()
```

실행결과

```
['최재원', '최지윤', '김연수', '김연우', '김가현', '김혜현']
search name : 김연수
010-3333-1234
```

실행결과

```
['최재원', '최지윤', '김연수', '김연우', '김가현', '김혜현']
search name : 최재운
not Found
```

5.7 컬렉션 자료형 도전문제

6. 동아리 A, B에 다음과 같이 학생들이 가입되어 있습니다. 실행결과를 참고하여 문제를 해결하시오.

```
clubA={'kim', 'park', 'hwang'}
clubB={'park', 'lee', 'choi'}
```

문제

1. 동아리에 가입한 학생들의 모든 명단을 ClubC에 저장한 후에 출력합니다.
2. A, B 동아리에 둘 다 가입한 학생의 명단을 출력하시오.
3. A 동아리에서 B 동아리에 가입한 학생들의 명단을 제외하고 출력하시오.
4. B 동아리에서 A 동아리에 가입한 학생들의 명단을 제외하고 출력하시오.
5. A 동아리에 새로 가입한 'yang' 회원을 추가하고 출력하시오.
6. B 동아리에서 탈퇴한 'lee' 회원을 삭제하고 출력하시오.
7. A, B 동아리 회원들을 각각 출력하시오.
8. 현재 남아 있는 A, B 동아리 회원들의 수를 각각 출력하시오.

조건 - 세트 자료구조를 사용합니다.

5.7 컬렉션 자료형 도전문제

실행결과

```
1 {'choi', 'hwang', 'park', 'lee', 'kim'}
2 {'park'}
3 {'hwang', 'kim'}
4 {'choi', 'lee'}
5 {'yang', 'hwang', 'kim', 'park'}
6 {'choi', 'park'}
7 clubA : {'yang', 'hwang', 'kim', 'park'}
7 clubB : {'choi', 'park'}
8 clubA 전체 인원수 4
8 clubB 전체 인원수 2
```

수고하셨습니다.